

Serveur de quiz QCM client/serveur en Python (sockets)

Objectif du TP

L'objectif de ce TP est de mettre en œuvre une communication client/serveur en Python à l'aide du module `socket`, sans utiliser de threads.

Le programme doit permettre :

- la création d'un serveur de quiz QCM
- la connexion d'un seul client
- l'envoi de 10 questions choisies aléatoirement depuis un fichier
- la réception des réponses du client (A/B/C/D)
- le calcul du score final
- la sauvegarde du score dans un fichier

Une attention particulière est portée à la qualité du code, à la découpe en fonctions et à l'utilisation correcte des fichiers avec `with open`.

Structure du projet

```
TP1/  
├─ server.py  
├─ client.py  
├─ questions.txt  
└─ scores.txt
```

Fonctionnement global de l'application

1. Le serveur démarre et attend une connexion
2. Le client se connecte au serveur
3. Le serveur sélectionne **10 questions aléatoires**
4. Pour chaque question :
 - le serveur l'envoie au client
 - le client répond (A/B/C/D)
 - le serveur vérifie la réponse
5. À la fin :
 - le serveur envoie le score final au client
 - le score est enregistré dans `scores.txt`

Explication du code serveur (`server.py`)

a) Chargement des questions

```
def load_questions():  
    """  
    Load questions from a file and return a list.  
    """
```

- Ouvre `questions.txt`
- Lit chaque ligne
- Découpe la ligne avec `split("|")`
- Stocke toutes les questions dans une liste

b) Envoi d'une question au client

```
def send_question(conn, question):  
    """  
    Send a formatted question to the client.  
    """
```

- Formate la question (sans la bonne réponse)
- Envoie la question via le socket avec `sendall()`

c) Réception de la réponse du client

```
def receive_answer(conn):  
    """  
    Receive the client's answer.  
    """
```

- Reçoit la réponse avec `recv()`
- Supprime les espaces
- Convertit la réponse en majuscule

d) Sauvegarde du score

python

```
def save_score(score):  
    """  
    Save the client score to a file.  
    """
```

- Ouvre `scores.txt` en mode ajout (`"a"`)
- Écrit le score final
- Utilise `with open` (bonne pratique R107)

e) Fonction principale du serveur

python

```
def start_server():  
    """  
    Start the quiz server.  
    """
```

- Crée le socket TCP
- Écoute sur `127.0.0.1:5000`
- Attend un client avec `accept()`
- Envoie les 10 questions
- Compare chaque réponse
- Calcule le score final
- Envoie le score au client
- Sauvegarde le score

Explication du code client (`client.py`)

6 Explication du code client (`client.py`)

a) Réception d'une question

python

 Copier le code

```
def receive_question(sock):  
    """  
    Receive a question from the server.  
    """
```

- Reçoit les données envoyées par le serveur
- Les affiche à l'écran

b) Envoi d'une réponse

python

 Copier le code

```
def send_answer(sock, answer):  
    """  
    Send the user's answer to the server.  
    """
```

- Envoie la réponse saisie par l'utilisateur au serveur

c) Fonction principale du client

python

 Copier le code

```
def start_client():  
    """  
    Start the quiz client.  
    """
```

- Se connecte au serveur
- Reçoit 10 questions
- Demande une réponse à l'utilisateur
- Envoie chaque réponse
- Affiche le score final reçu du serveur

Comment exécuter le programme

Étape 1 : lancer le serveur

Dans un premier terminal :

1. `cd TP1`
2. `python3 server.py`
Server waiting for connection...

```

• ruhollah_sherzad@h111-14:~/Documents/S3/309/cours$ cd TP1
• ruhollah_sherzad@h111-14:~/Documents/S3/309/cours/TP1$ ls
client.py questions.txt scores.txt server.py
• ruhollah_sherzad@h111-14:~/Documents/S3/309/cours/TP1$ server.
py
server.py : commande introuvable
• ruhollah_sherzad@h111-14:~/Documents/S3/309/cours/TP1$ python3
server.py
Server waiting for connection...
Client connected: ('127.0.0.1', 39814)
• ruhollah_sherzad@h111-14:~/Documents/S3/309/cours/TP1$ █

```

lancer le client

Dans un **second terminal** :

1. `cd TP1`
2. `python3 client.py`

Le quiz s'affiche, l'utilisateur répond aux questions.

```

• ruhollah_sherzad@h111-14:~/Documents/S3/309/cours/TP1$ ls
client.py questions.txt scores.txt server.py
• ruhollah_sherzad@h111-14:~/Documents/S3/309/cours/TP1$ python3 client.py

What does TCP ensure?
A:Speed
B:Encryption
C:Reliability
D:Broadcast
Your answer (A/B/C/D): c

```

vérification du score

Après la fin du quiz :

```

• ruhollah_sherzad@h111-14:~/Documents/S3/309/cours/TP1$ cat scores.txt
Score: 7/10
• ruhollah_sherzad@h111-14:~/Documents/S3/309/cours/TP1$ █

```

Code complete [Server.py](#)

```
import socket
import random

HOST = "127.0.0.1"
PORT = 5000
QUESTIONS_FILE = "questions.txt"
SCORES_FILE = "scores.txt"

def load_questions():
    """
    Load questions from a file and return a list.
    """
    questions = []
    with open(QUESTIONS_FILE, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            questions.append(parts)
    return questions

def send_question(conn, question):
    """
    Send a formatted question to the client.
    """
    msg = "\n".join(question[:-1])
    conn.sendall(msg.encode())

def receive_answer(conn):
    """
    Receive the client's answer.
    """
    return conn.recv(1024).decode().strip().upper()

def save_score(score):
    """
    Save the client score to a file.
    """
    with open(SCORES_FILE, "a") as f:
```

```

        f.write(f"Score: {score}/10\n")

def start_server():
    """
    Start the quiz server.
    """
    questions = load_questions()
    quiz = random.sample(questions, 10)
    score = 0

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind((HOST, PORT))
        s.listen(1)
        print("Server waiting for connection...")

        conn, addr = s.accept()
        with conn:
            print("Client connected:", addr)

            for q in quiz:
                send_question(conn, q)
                answer = receive_answer(conn)
                if answer == q[-1]:
                    score += 1

            conn.sendall(f"Final score: {score}/10".encode())
            save_score(score)

if __name__ == "__main__":
    start_server()

```

Code complete [Client.py](#)

```
import socket

HOST = "127.0.0.1"
PORT = 5000

def receive_question(sock):
    """
    Receive a question from the server.
    """
    return sock.recv(1024).decode()

def send_answer(sock, answer):
    """
    Send the user's answer to the server.
    """
    sock.sendall(answer.encode())

def start_client():
    """
    Start the quiz client.
    """
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))

        for _ in range(10):
            question = receive_question(s)
            print("\n" + question)
            answer = input("Your answer (A/B/C/D): ").upper()
            send_answer(s, answer)

        result = s.recv(1024).decode()
        print("\n" + result)

if __name__ == "__main__":
    start_client()
```

Questions.txt

```
What is the default HTTP port?|A:21|B:22|C:80|D:443|C
What protocol assigns IP addresses?|A:DNS|B:DHCP|C:FTP|D:HTTP|B
What does TCP ensure?|A:Speed|B:Encryption|C:Reliability|D:Broadcast|C
What is the loopback IP?|A:192.168.1.1|B:10.0.0.1|C:127.0.0.1|D:8.8.8.8|C
What layer is IP?|A:Application|B:Transport|C:Network|D:Physical|C
What does DNS do?|A:Routing|B:Name resolution|C:Encryption|D:Switching|B
Which is a private IP?|A:8.8.8.8|B:1.1.1.1|C:192.168.0.1|D:54.23.12.1|C
What protocol uses port 22?|A:FTP|B:SSH|C:HTTP|D:SMTP|B
What is UDP?|A:Reliable|B:Connection-oriented|C:Fast|D:Encrypted|C
What device works at layer 2?|A:Router|B:Switch|C:Firewall|D:Server|B
```

Fome des question

Format:

ess

 Copier le code

question|A:...|B:...|C:...|D:...|CorrectLetter